React Native and TypeScript Study Guide

Concepts, reasoning, visual explanations, and a practical study plan

Independently prepared study materials based on the ten supplied Autumn 2026 lecture decks. Sources checked 6 November 2026. The practice questions and rubrics are original and are not official assessments.

Learn to explain what your code does before trying to memorize every spelling. Five core skills connect much of the material: arrow functions, returned values in map, immutable array updates, callback props, and TypeScript inference. Choose the idea that feels unclear, or use the contents to explore any lecture topic.

Use this guide as a reference in any order. Read a short explanation, trace an example, and try a matching exercise when you are ready. The detailed six-hour schedule is an optional appendix at the end; you do not need to follow it to use these materials.

Open tutorials/index.html for the interactive study hub. Each core skill offers Understand, Example, Try it, and Self-check views. The site also has visual labs, searchable practice, saved scratchpads, and mock interviews with an optional timer. The separate Practice Workbook contains exercises and answer keys.

Contents

Understand the data before writing the screen

Make arrow functions feel predictable

Read a screen as a tree of components

Understand state as a sequence of renders

Turn an array into a visible list

Build the next array without changing the old one

Connect a form to its parent with callback props

Reason about spacing and Flexbox axes

Know what React Native and Expo provide

Organize files and recover from errors aloud

Use hooks and timers without creating hidden repeats

Follow asynchronous results without blocking the screen

Five core skills to explore

These five skills connect much of the coding material. Choose one that needs attention, trace its JavaScript behavior, and add TypeScript where it clarifies the contract. Return to any skill whenever a practice task reveals a gap.

1 Read and write arrow functions

An arrow function is a value that can be passed to another function or component. Its parameter receives an argument later. An expression body returns a value automatically; a block needs return when a value is required.

Practice E01. Write both arrow forms from memory, return an object correctly, and explain the input and return value.

2 Turn an array into visible components with map

map calls the callback once per item and collects its returned values. A missing return produces undefined entries. Store the returned element array in a variable or place the expression inside JSX braces. Give each rendered sibling a stable key.

Practice E02. Render a small list in both arrow styles, explain item versus items, and fix a missing return without guessing.

3 Construct a new array for a state update

Square brackets make an array, and ...previous copies its entries into that new array. A setter accepts either the next value or a pure function that returns it. For an append, setMenu(previous => [...previous, item]) is valid and uses the previous queued state.

Practice E04. Append without push, show both valid setter forms, and distinguish array spread from object spread and a function block.

4 Pass a parent callback to a child

Put shared state in the parent. Pass the parent's update function as a prop. The child calls it in an event handler with the required argument. The parent updates state and the next render sends the new values down again.

Practice E07. Build the parent and child from a blank file. Explain press -> child handler -> parent callback -> setter -> new render. Then adapt the string argument to a { name, popular } object.

5 Use inference first and add types deliberately

useState(initialMenu) can infer MenuItem[] when initialMenu already has that type. Empty arrays and null initial values often need explicit type arguments. Callback props need a clear function contract. TypeScript checks your code; it does not change the JavaScript callback behavior.

Practice E08. Explain why one state call can infer its type while useState<MenuItem[]>([]) and useState<MenuItem | null>(null) need guidance.

Before moving to breadth, complete exercises e01, e02, e04, e07, and e08 without opening their solutions. Then attempt mock m01. If you need a hint, record the exact step, repeat it later with different data, and explain the repair aloud. This is study evidence, not a guarantee about an assessment.

Four callbacks with different jobs

map callback

Receives: One array item. Returns: A transformed value, often a React element.

```
items.map(item => <ItemCard key={item.id} item={item} />)
```

State updater

Receives: The previous queued state. Returns: The complete next state value.

```
setMenu(previous => [...previous, newItem])
```

Event handler

Receives: Event arguments supplied by the component, which you may ignore. Returns: Its return value is not used as new state.

```
onPress={() => onAdd({ name, popular })}
```

Callback prop

Receives: The argument the child passes, according to the prop type. Returns: Usually void for an action callback.

```
type Props = { onAdd: (item: MenuDraft) => void }
```

How to study

Read one example, hide it, and reproduce it from a blank file. Predict the UI or returned value, run the code, and explain a mismatch. Repeat the pattern later with different data. A correct copied answer is weaker evidence than a small original solution you can explain.

Readiness checks

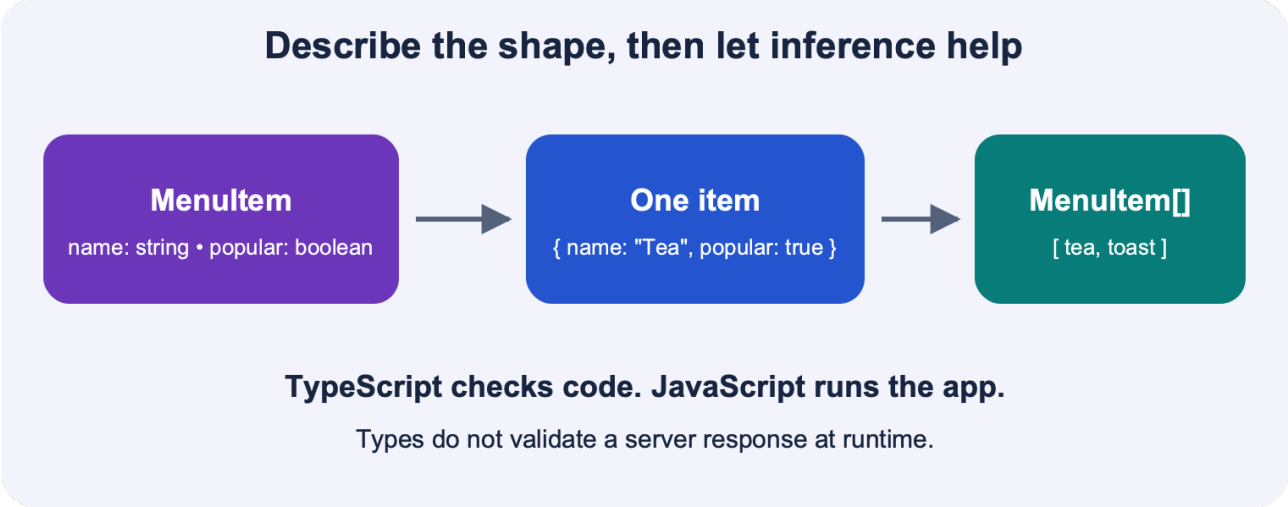
- Without hints, write a typed arrow function in expression and block forms and explain the return value of each.
- From a blank fragment, render a typed data array through a custom component with a correct map return and stable keys.
- Explain and write both valid setter forms. Append twice through functional updates without losing either item.
- Toggle one object without mutating the old array or old object, and remove by ID with a correct predicate.
- Build a controlled TextInput and parent-owned list connected through a typed callback without looking at the integrated solution.
- Restate a callback’s input object shape before constructing it, and explain why the handler runs on a press rather than during rendering.

- Use inference for useful initial values and explicit types for empty arrays, nullable state, and standalone function parameters.
- Complete the core of a 25-minute original mock, record any hints, and repeat the key skill later without those hints. A practice score tracks learning; it is not an official course result.
- Pass a transfer attempt using new fields and requirements, such as tasks with completion instead of menu items with popularity.
- Explain nested Flexbox axes, core component choices, box-model spacing, and default versus named imports.
- Predict state batching, effect cleanup, and Promise log order, then confirm the prediction with a focused experiment.
- Use the answer key and code cheat sheet after attempting the problem. Close them before a timed reconstruction or transfer task.

Understand the data before writing the screen

Essential / Lecture 2a slides 4-6, 12-14, 40-44

- Describe the difference between a value and its type.
- Read object, array and function types without guessing.
- Use inference when it is informative and annotations when they are needed.



The MenuItem type describes an object. An array contains several objects. Type checking happens before runtime.

JavaScript runs the program and TypeScript checks its shape

The code that runs on the device is JavaScript. TypeScript adds a system for describing the kinds of values your program expects. A number can be added to another number; a menu item is an object with named fields; an array is an ordered collection. A type checker can catch many mismatches before you run the app. It cannot prove that your layout is attractive, that a timer is cleaned up, or that an answer from a server is trustworthy.

Think in two passes when you are under time pressure. First describe the operation in plain JavaScript: “Make a new array that contains the old items and this new item.” Then check the boundary types: “The input is one MenuItem; the state is an array of MenuItem.” This separates the operation from the type

annotation. It does not mean deleting every type annotation or silencing errors with any. TypeScript type annotations are normally removed when the project is transformed into JavaScript.

Pause and predict: If a variable is declared as number, does TypeScript convert the text "12" into the number 12?

Explanation: No. Types describe expectations; they do not perform conversions. Number("12") performs a runtime conversion. A numeric keyboard also does not change TextInput text into a number.

Read an object and an array as different structures

An object groups fields that describe one thing. An array groups many things in a particular order. In this guide, a MenuItem has an id, a name and a popular flag. The id provides stable list identity; some smaller examples need only name and popular. Keep the required shape consistent within any one exercise.

Use item.name when you want a field of one object. Use menu[0] when you want the first element of an array. Use menu.length when you want the number of elements. A type ending in [] means an array of values of that type. MenuItem[] is not the same shape as MenuItem, and an object with numeric property names is not an array.

Dot notation and bracket notation can read the same property: item.name and item["name"] both read the name field. Bracket notation is also useful when a variable holds the property name, provided TypeScript knows that key is valid. A const binding cannot be reassigned, but const does not freeze an object or array; you still need the immutable-update discipline for React state.

```
// Standalone TypeScript data example.
type MenuItem = {
  id: string;
  name: string;
  popular: boolean;
};
const item: MenuItem = { id: 'tea', name: 'Tea', popular: true };
const menuItems: MenuItem[] = [item];
console.log(item.name);        // Tea
console.log(menuItems.length); // 1
console.log(menuItems[0].popular); // true
```

Common trap: Square brackets create an array. Curly braces create an object in expression position. Changing the brackets changes the data structure, not just the appearance of the code.

Let useful initial values do some of the typing

Inference means TypeScript works out a type from evidence in the surrounding code. If menuItems is already declared as MenuItem[], useState(menuItems) can infer the state type. Similarly, useState(0) gives number state and useState("") gives string state. Repeating the same type is allowed, but it is not always necessary.

An empty array does not tell the checker what future elements will look like. In a strict React TypeScript project, useState([]) commonly infers never[], which rejects added objects. Supply useState<MenuItem[]>([]) when you start empty. If a selected item may be absent initially and later hold an object, write useState<MenuItem | null>(null). The vertical bar means either alternative is permitted. You still need to check for null before reading its properties.

```
// Fragment inside a component; import useState from react.
// MenuItem and menuItems are defined in the preceding example.
const [menu, setMenu] = useState(menuItems); // inferred MenuItem[]
const [emptyMenu, setEmptyMenu] = useState<MenuItem[]>([]);
const [selected, setSelected] = useState<MenuItem | null>(null);
const title = selected === null ? 'Choose an item' : selected.name;
```

Pause and predict: Why is useState<MenuItem[]>([]) more useful than useState<any[]>([])?

Explanation: The explicit element type tells TypeScript what an item must contain. any allows mistakes such as appending a number or omitting a required field, so it removes a check that would help you debug.

Use object destructuring and shorthand deliberately

Destructuring reads named fields from an existing object into local variables. In const { name, popular } = item, no new menu item is created: name and popular are local bindings to values already stored on item. Object shorthand goes the other direction. If local variables are called name and popular, { name, popular } constructs the same fields as { name: name, popular: popular }.

Props often use destructuring because a component receives one props object. Writing function ItemCard({ item }: ItemCardProps) means “take the item field from the props object, and check the whole props object against ItemCardProps.” It does not mean that item itself has the shape ItemCardProps. Say this sentence aloud before writing callback props; it prevents a common confusion between an item and a wrapper containing an item.

```
// Standalone TypeScript example.
const source = { name: 'Rice', popular: false };
const { name, popular } = source; // read two fields
const copy = { name, popular };  // construct a new object
console.log(copy.name); // Rice
```

Read function types as promises between pieces of code

The type (item: MenuItem) => void says the caller must supply one MenuItem and should not use a return value. It is a good type for an onAdd prop whose job is to request a state change. By contrast, (item: MenuItem) => string describes a transformation that returns a string. These functions may take the same input but serve different roles.

Annotate parameters of standalone functions when TypeScript has no context from which to infer them. In menu.map(item => item.name), the menu array already supplies the parameter type. In const label = item => item.name, the checker may have no such evidence. Do not assume inference means all parameter annotations are unnecessary. The word void describes a contract about ignoring a result; it is not a runtime command.

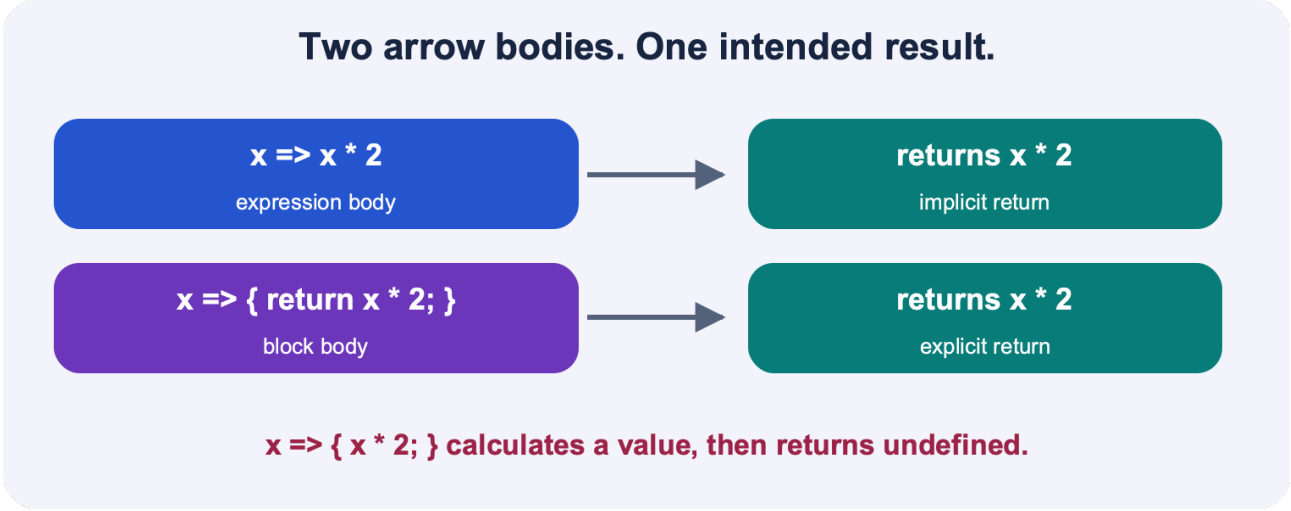
```
// Type declarations; MenuItem is defined above.
type AddHandler = (item: MenuItem) => void;
type LabelMaker = (item: MenuItem) => string;
const label: LabelMaker = item => item.name;
const describe = (item: MenuItem): string => item.name;
```

Read more: TypeScript The Basics / TypeScript Everyday Types / TypeScript Type Inference / React Using TypeScript

Make arrow functions feel predictable

Essential / Lecture 2a slides 14, 40-44; Lecture 3a slides 5-6; Lecture 3b slides 9-11

- Distinguish a function value from a function call.
- Translate an expression body into a block body with return.
- Recognize which callback arguments a component supplies.



An expression body returns its result. A block body needs an explicit return. A block without return yields undefined.

A function is a value that can be saved and passed

An arrow function describes work that can be done later. The parameter is a placeholder for the value supplied when the function is called. In item => item.name, item is not the whole menu. It is whichever individual item the caller supplies on a particular call. The name is your choice; entry => entry.name would do the same work.

You can store a function in a variable, pass it to another function, or pass it through a component prop. A callback is simply a function passed to some other code so that code can call it. map calls its callback while producing the new array. A Pressable calls its handler after a press. “Callback” tells you the relationship between the functions; it does not automatically mean asynchronous.

```
// Standalone TypeScript example.
const double = (value: number) => value * 2;
console.log(typeof double); // function
console.log(double(4));     // 8
const saved = double;       // save the function itself
console.log(saved(5));      // 10
```

The two return forms have the same result

With an expression body, the expression after => becomes the return value. With a block body, braces introduce a sequence of statements, and you must write return if you want a result. Parentheses around an expression can make a multiline JSX expression easier to read; they are not the thing that makes it return. The absence of a block body is what gives the implicit return.

Before debugging a map, point at the arrow and ask, “Does this arrow return an expression, or open a block?” If it opens a block, find the return statement. The line breaks do not determine the answer. A one-line function with braces still needs return, and an expression-body function may span several lines.

```
// Standalone TypeScript example.
const a = (x: number) => x * 2;
const b = (x: number) => { return x * 2; };
const c = (x: number) => { x * 2; }; // intentionally wrong for mapping
console.log(a(3)); // 6
console.log(b(3)); // 6
console.log(c(3)); // undefined
```

Common trap: Putting return on a line by itself can trigger JavaScript automatic semicolon insertion. Write return (on the same line when returning multiline JSX.

Pause and predict: Does x => { x + 1; } return x + 1?

Explanation: No. It evaluates x + 1 and discards the result. Use x => x + 1 or x => { return x + 1; }.

Return an object by making the expression explicit

Curly braces directly after an arrow are read as a function body. To return an object with the expression form, wrap the object literal in parentheses: name => ({ name }). A block with an explicit return works too. This resolves the shortened-syntax puzzle raised in Lecture 2a: an arrow can return an object, but bare braces are ambiguous in that position.

Arrays do not have this ambiguity. item => [item] clearly returns an array. This distinction is important when constructing new state. prev => [...prev, item] returns a new array; prev => ({ ...prev, item }) returns an object; prev => { ...prev, item } is not a valid array update.

```
// Standalone TypeScript example.
const makeA = (name: string) => ({ name, popular: false });
const makeB = (name: string) => {
  return { name, popular: false };
};
console.log(makeA('Soup').popular); // false
console.log(makeB('Soup').name);    // Soup
```

Pass a handler now and let the event call it later

If a handler already accepts the arguments you need, pass it directly. onPress={handlePress} passes a function. onPress={handlePress()} calls that function while rendering and passes its return value, which is usually undefined. If the call changes state, doing it during render can cause repeated renders.

Use a wrapper when you need to provide your own argument. onPress={() => onAdd(item)} gives Pressable a no-argument wrapper whose body calls onAdd with a MenuItem. Pressable supplies a press event to its handler; it does not know that your application wants to add a menu item. The wrapper translates from a UI event to your application action.

```
// JSX fragments; handlePress, onAdd and item are already defined.
<Pressable onPress={handlePress}><Text>Open</Text></Pressable>
<Pressable onPress={() => onAdd(item)}><Text>Add</Text></Pressable>
// Avoid: onPress={onAdd(item)}. That calls onAdd while rendering.
```

Trace the values

- Render: create a wrapper function and give it to Pressable.
- Press: Pressable calls the wrapper.
- Wrapper: call onAdd(item) with the intended item.
- Parent handler: request the state update.

Practice the punctuation without memorizing a screenshot

Write a function that doubles a number in both forms, then a function that returns an object in both forms. Next write the same transformation as a map callback. Finally place a handler in onPress. The words and data should change each repetition so that you learn the roles of the brackets rather than the appearance of one example.

When explaining your code, name the input, the time of the call and the result. For example: “map calls this function once for each menu item now, and it returns a card element.” Then: “Pressable calls this other function later, and it asks the parent to add an item.” These short explanations expose misunderstandings before they become a screen full of syntax errors.

Pause and predict: Is every arrow function a callback?

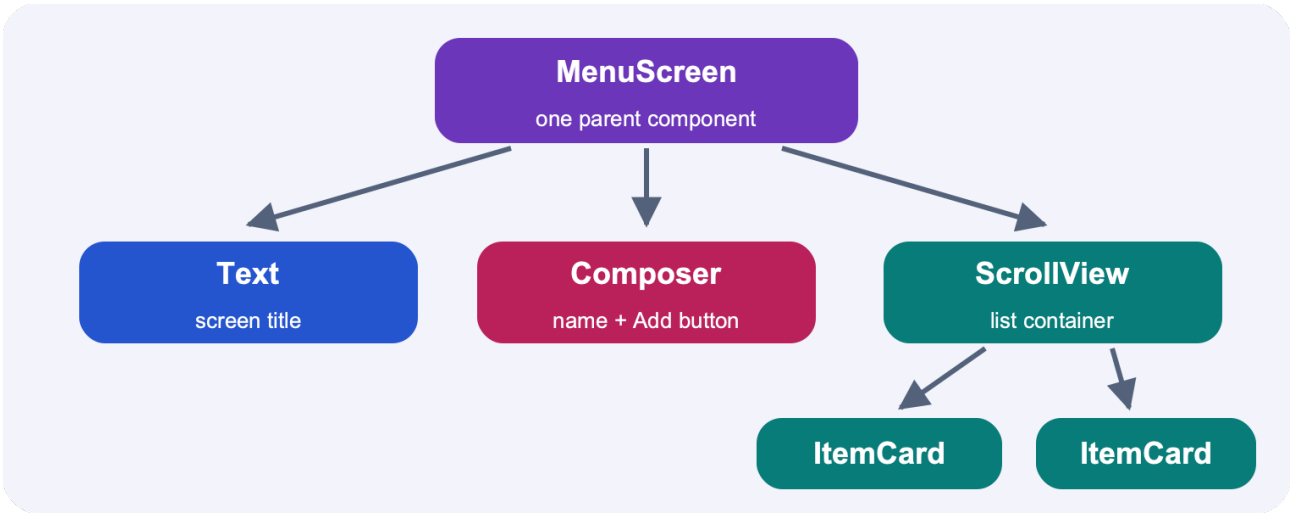
Explanation: No. An arrow is a way to write a function. It is a callback when you pass it to other code to be called by that code.

Read more: TypeScript More on Functions / React Rendering Lists

Read a screen as a tree of components

Essential / Lecture 1b slide 8; Lecture 2a slides 13, 15-21, 34-38; Lecture 3b slides 15-16, 26; Lecture 4a slides 15-16, 26

- Choose a suitable React Native component.
- Pass data through typed props.
- Use JavaScript expressions and conditional rendering inside JSX.



MenuScreen contains a title, a composer, and a list. The list has two ItemCard components. Props specialize reusable components.

JSX describes what the interface should contain

A component is a function that returns a description of UI. React calls it to work out what should be displayed. JSX lets you express that description with nested tags. The nesting shows a tree: a View may contain a heading Text and several item cards. JSX is JavaScript syntax, not an HTML page embedded in your app. In TypeScript files that contain JSX, use the .tsx extension.

Tags with capitalized names refer to components. Import View and Text from react-native rather than writing web tags such as div or p. Close every tag. A component can return one enclosing element or a fragment, written <>...</>, containing siblings. A fragment groups elements without creating another native layout box.

```
// Complete small React Native component file.
import { Text, View } from 'react-native';
export default function Welcome() {
  const name = 'CS 147L learner';
  return (
    <View>
      <Text>Welcome, {name}</Text>
      <Text>Explain your next step aloud.</Text>
    </View>
  );
}
```

Use braces to insert an expression

In JSX, quoted text is a literal string. A pair of braces lets you insert a JavaScript expression: a variable, a function value, a calculation, or the result of map. For a prop, title="Menu" passes a fixed string, while title={screenTitle} reads the value of a variable. style={{ padding: 16 }} contains two different pairs of braces: the outer pair enters JavaScript, and the inner pair creates a style object.

You cannot place an if statement directly inside JSX braces because if is a statement, not an expression. Compute a variable before return, use a ternary expression, or use a boolean condition with &&. Keep the condition actually boolean; 0 && <Text>Items</Text> evaluates to 0, so count > 0 is clearer and avoids accidentally rendering a number.

```
// JSX fragment inside a component with menu state.
<View>
  <Text>{menu.length} items</Text>
  {menu.length === 0 ? <Text>No items yet.</Text> : <Text>Ready to
order.</Text>}
  {menu.length > 0 && <Text>Choose a favorite.</Text>}
</View>
```

Pause and predict: Why are there two opening braces in style={{ padding: 16 }}?

Explanation: The outer braces switch from JSX into a JavaScript expression. The inner braces are the object literal that holds the padding property.

Choose core components for their jobs

View is a general container for layout and backgrounds. Text displays text and supports nested Text when part of a sentence needs different styling. Image displays an image; remote images usually need an explicit width and height. ScrollView provides scrolling for a modest amount of content. Button gives a standard button through its title and onPress props. Pressable gives you control over the visual content and press response.

TextInput accepts text, ActivityIndicator communicates loading, and Alert.alert opens a platform alert where supported. Alert is an API you call, not a component that you put in JSX. Do not hide a broken interaction behind decorative styling. First verify that a button invokes the handler and that a TextInput updates state, then make them look good. For very long lists, FlatList is a useful extension because it avoids rendering every row at once; it is not a substitute for understanding map.

Nested Text can inherit text styling from an enclosing Text. In the example, the inner word keeps the outer bold weight and changes its color. This limited text inheritance differs from expecting a font style on a View to cascade through an arbitrary component tree.

```
// JSX fragment; import Image and Text from react-native.
<Image source={{ uri: 'https://reactnative.dev/img/tiny_logo.png' }}
  style={{ width: 48, height: 48 }} accessibilityLabel="React Native logo" />
<Text style={{ fontWeight: 'bold' }}>
  Menu <Text style={{ color: '#165DCE' }}>today</Text>
</Text>
```

A prop is an input to a reusable component

An ItemCard should not need to know where its item came from. Give it an item prop, and it can display one item from any appropriate list. Define the prop type, destructure the props object, and read its fields. Props are read-only inputs. A child should not change item.popular directly because that object may belong to its parent’s state.

Custom components are useful when a visual pattern repeats or a piece of behavior has a clear responsibility. Extracting every single Text into a component can make a small exercise harder to follow. A good interview explanation is, “The parent owns the menu array; this card displays one item.” If the card needs to request a change, add a callback prop instead of giving it a second copy of the same state.

```
// Complete reusable component file.
import { Text, View } from 'react-native';
type MenuItem = { id: string; name: string; popular: boolean };
type ItemCardProps = { item: MenuItem };
export default function ItemCard({ item }: ItemCardProps) {
  return (
    <View style={{ padding: 12 }}>
      <Text>{item.name}</Text>
      {item.popular && <Text>Popular</Text>}
    </View>
  );
}
```

Common trap: key is a special React list attribute, not an ordinary prop delivered to the child. Pass item.id or a separate id prop if the child needs that value.

Make disabled and empty states explicit

Conditional UI is part of the application behavior. A sold-out button should be disabled as well as visually different. A list with zero items should explain the empty state. A loading operation should give feedback rather than leaving an apparently frozen screen. Use the same underlying state to determine both the label and the interaction so they cannot contradict each other.

When testing a screen, try the boundary values: zero items, one item, a long name and several repeated presses. These often reveal a mistaken condition or missing layout constraint faster than the ordinary case. Explain the reason for your condition instead of reading it aloud: “The action requires stock, so the button is disabled when stock is zero or less.”

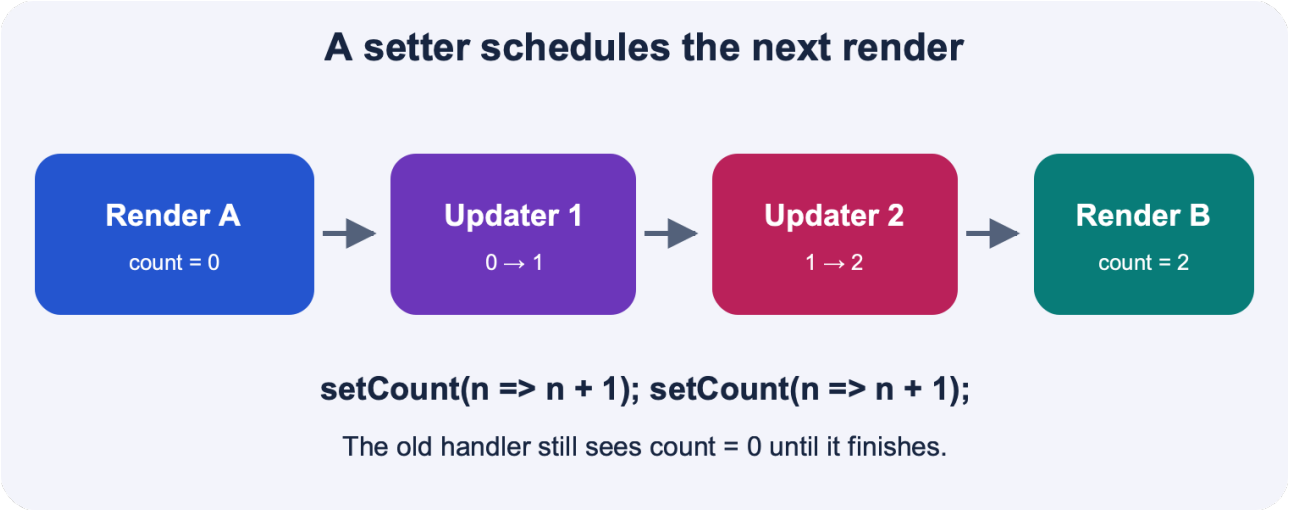
```
// JSX fragment; stock and buy are defined by the component.
<Button
  title={stock > 0 ? 'Buy one' : 'Sold out'}
  disabled={stock <= 0}
  onPress={buy}
/>
```

Read more: React Native Core Components and APIs / React Using TypeScript

Understand state as a sequence of renders

Essential / Lecture 3a slides 8-12; Lecture 3b slides 13-14; Lecture 5a slide 10

- Explain why a local variable cannot drive a persistent interactive screen.
- Predict direct and functional updates.
- Keep state update functions pure.



The current render reads count zero. Two queued functional increments calculate one then two. A new render receives two.

A render receives a snapshot

Each time React calls a component, the function reads the state values for that render and produces UI. Ordinary local variables are recreated when the function runs again. Changing a local variable also does not tell React to update the screen. A module-level variable may survive function calls, but changing it still does not notify React, and it can be accidentally shared by multiple component instances.

useState gives you the current value and a setter. The setter asks React to compute a later render with a new state value. It does not rewrite the local binding in the currently running handler. That is why const [count, setCount] is correct: count itself is not reassigned. A later call to the component gets a new count binding.

```
// Complete small React Native component file.
import { useState } from 'react';
import { Button, Text, View } from 'react-native';
export default function Counter() {
  const [count, setCount] = useState(0);
  return (
    <View>
      <Text>{count}</Text>
      <Button title="Add one" onPress={() => setCount(c => c + 1)} />
    </View>
  );
}
```

A setter accepts a value or a pure updater function

React permits both a next value and an updater function. Lecture 5a slide 10 teaches both forms. Passing an arrow function to a setter is therefore valid when that function computes the next state. If an array update fails, inspect what the function returns before concluding that the function itself is forbidden.

For array state, setMenu([...menu, item]) is a valid direct update. setMenu(previous => [...previous, item]) is a valid functional update and is the clearer default when the new value depends on the previous value. The updater receives the pending state and must return the next state of the same type. It should not send a message, increment an external ID counter, or modify the old array.

```
// Fragments inside a component; menu is MenuItem[] and item is MenuItem.
setMenu([...menu, item]); // direct replacement using this render's menu
setMenu(previous => [...previous, item]); // replacement from pending state
// These are alternatives. Do not execute both to add one item.
```

Pause and predict: Was the idea of passing menu => ... to the setter inherently wrong?

Explanation: No. Passing a pure updater is supported. The updater must return the correct next array, for example previous => [...previous, item].

Predict two queued updates before trying them

Suppose the current render has count equal to 0. Two calls to setCount(count + 1) both calculate the number 1 from that same snapshot. They request the same replacement value, so the next count is 1. Two calls to setCount(c => c + 1) queue two transformations. The second receives the result of the first, so the next count is 2.

This is a reasoning exercise about values and functions. You do not need to memorize a vague rule that setters are “slow.” Ask whether each call supplies a value computed now or a function to compute from pending state. React normally batches updates during a handler, which is why several setter calls do not mean several immediate visible screen changes.

```
// Two alternative event handlers when the current count is 0.
const replaceTwice = () => {
  setCount(count + 1);
  setCount(count + 1);
}; // next count: 1
const incrementTwice = () => {
  setCount(c => c + 1);
  setCount(c => c + 1);
}; // next count: 2
```

Trace the values

- Direct case: calculate 0 + 1 twice, then queue replace-with-1 twice.
- Updater case: start at 0, apply +1 to get 1, then apply +1 to get 2.
- In both handlers, reading count immediately still reads the current snapshot.

Store the minimum information that needs memory

If a value can be calculated from current props or state, usually calculate it while rendering. For example, menu.length already tells you the number of items. Keeping a separate itemCount state creates a second value that must remain synchronized. You could add an item correctly but forget to update the count.

Use separate state for independent facts, such as the draft name currently being typed and the list of saved items. When one is derived from another, prefer a calculation. This also makes the interview explanation simpler: “The displayed total is computed from the array, so it always matches.” Effects are not required to keep a derived count in sync.

```
// Fragment inside a component that already owns menu state.
const itemCount = menu.length;
const popularCount = menu.filter(item => item.popular).length;
// Both are recalculated from the current menu during rendering.
```

Read old logs and development repeats correctly

If you log count immediately after setCount, the log shows the count from that render. To predict the next value for a simple direct update, compute it in a local variable before calling the setter. Do not try to await setCount: it does not return a Promise that resolves when rendering finishes.

In development Strict Mode, React can call component bodies or updater functions extra times to expose accidental side effects. A pure updater still has the same intended result because it only calculates a value. If a repeated development call duplicates a network request or changes an external counter, the function is doing more than a state calculation. Fix that impurity rather than relying on the updater being called exactly once.

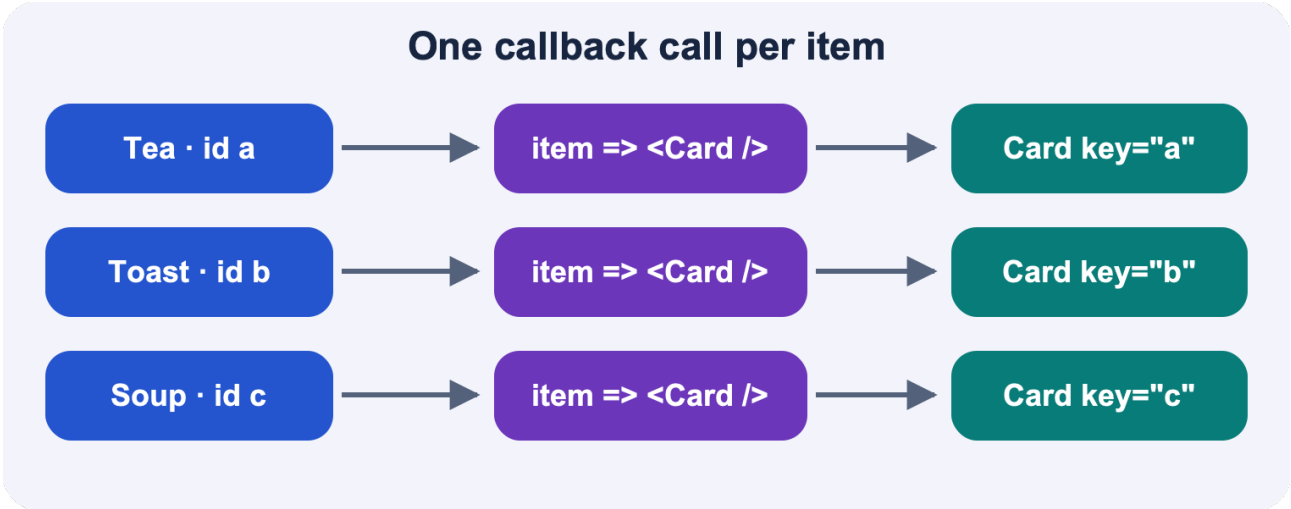
Common trap: The sample counter on Lecture 3a slide 11 and Lecture 3b slide 13 uses a setter name that does not match its declaration. In your code, use setCount when the hook declares [count, setCount].

Read more: React useState / React Queueing a Series of State Updates

Turn an array into a visible list

Essential / Lecture 3b slide 11; Lecture 4a slide 13; Lecture 4b slide 4

- Trace the input and result of map.
- Return one element for each item.
- Choose stable keys and distinguish map from filter.



Three data items are each passed to a callback and become three elements. Each rendered sibling needs a stable key.

Map transforms values and returns a new array

For each present element in an ordinary dense array, map calls your function and collects its return value. The callback receives the current element, and may also receive its index and the original array. In [2, 4, 8].map(n => n * 3), the callback is called with 2, then 4, then 8. The returned values become [6, 12, 24]. The original numbers remain unchanged.

The transformation can return numbers, strings, objects or React elements. This is why map is useful for rendering: your data array is the input, and an array of element descriptions is the result. map does not display anything by itself. React sees those elements only when you include the resulting array in the JSX that the component returns.

```
// Standalone TypeScript example.
const prices = [2, 4, 8];
const labels = prices.map(price => $${price.toFixed(2)});
console.log(labels); // ['$2.00', '$4.00', '$8.00']
console.log(prices); // [2, 4, 8]
```

Write both rendering forms and explain the return

The expression form is convenient for a simple card. The block form is useful when you want to calculate a label or inspect a value before returning the element. Both forms produce the same kind of result. In either form, return the element you want React to render. A block that contains JSX but has no return produces undefined for that item.

You may assign the result to a variable or place map directly inside JSX braces. Start with a named variable if it helps you separate transformation from display. Once that is comfortable, the inline version is a shorter way to write the same idea.

```
// Fragment inside a component; menu and ItemCard are already defined.
const cards = menu.map(item => (
  <ItemCard key={item.id} item={item} />
));
const sameCards = menu.map(item => {
  return <ItemCard key={item.id} item={item} />;
});
// Render one of these arrays, not both:
return <ScrollView>{cards}</ScrollView>;
```

Pause and predict: Why can a list contain no visible cards even though menu.length is 3?

Explanation: One possible cause is a block-body map callback with no return. It produces three undefined values, so React has no card elements to display. Also check that the result is included in returned JSX.

Use a stable key to preserve identity

React uses key to match sibling list elements across renders. A stable unique item ID lets React recognize that an existing item has moved or that a different item was deleted. Place the key on the outermost element returned by the map callback. If you return a View containing ItemCard, the View needs the key.

An array index can be misleading when items are inserted, removed or reordered because the same position may now refer to a different item. A random value generated during rendering is worse: it changes identity every render. For these exercises, assign IDs when data is created and keep them with the items. Names are safe keys only when you can guarantee that they are unique and never change.

```
// JSX fragment showing the key on the returned outer element.
{menu.map(item => (
  <View key={item.id} style={{ marginBottom: 8 }}>
    <ItemCard item={item} />
  </View>
))}
```

Common trap: A warning about a missing key is different from a missing-return bug. Adding a key does not make an undefined return value become a card.

Separate selection from transformation

filter answers, “Which elements should remain?” Its callback returns true to keep an item and false to leave it out. map answers, “What value should each element become?” Its callback returns the replacement value. To show only popular items, filter first and then map the selected items into cards.

Neither method should mutate objects stored in state. map returns a new outer array, but a callback can still modify an existing item if you write item.popular = true. A safe transformation returns a new object for a changed item. The next lesson explains why both the array and the changed object need attention.

```
// JSX-producing fragment; menu and ItemCard are defined.
const popularCards = menu
  .filter(item => item.popular)
  .map(item => <ItemCard key={item.id} item={item} />);
```

Debug a list by following the data once

First inspect the array: is it empty, and do the fields have the names you expect? Next inspect the callback: does it return the intended element? Then inspect the returned JSX: is the result actually inserted? Finally check the component and layout: is the text visible, and can the screen scroll? This sequence tests distinct possibilities instead of repeatedly changing punctuation at random.

Explain one item all the way through the process. “This object has name Tea. map passes that object to the callback. The callback returns ItemCard with that object as its item prop. ItemCard renders item.name inside Text.” If you can narrate that chain without looking at the answer, you understand the list rather than recognizing a snippet.

Pause and predict: What does [1, 2].map(x => { x * 2; }) produce?

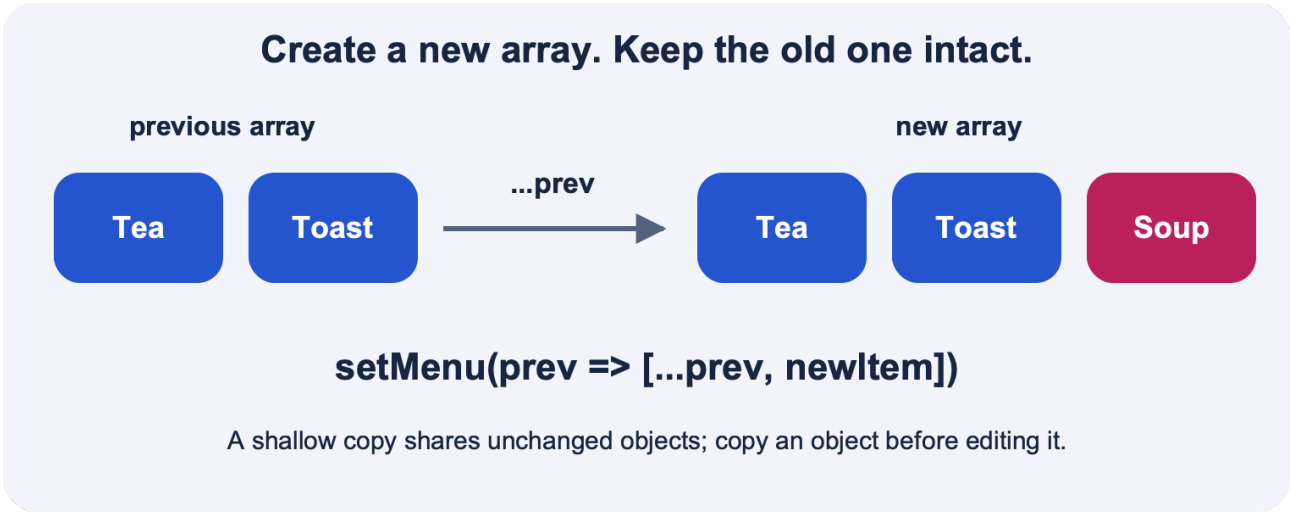
Explanation: It produces [undefined, undefined]. The block evaluates multiplication but never returns its result.

Read more: React Rendering Lists

Build the next array without changing the old one

Essential / Lecture 4b slides 5-8; Lecture 5a slide 10

- Choose array or object spread according to the value shape.
- Append, insert, replace and remove without mutation.
- Explain shallow copies and state reference checks.



The old array holds Tea and Toast. A new array holds the old entries plus Soup. The outer array is new; unchanged objects can be shared.

Start with the shape of the next state

If menu state holds MenuItem[], the next state must also be an array of menu items. The expression [...previous, item] creates that array: it expands the old elements into a new array and puts the new item after them. The surrounding square brackets are essential. The spread operator does not decide the container type; the surrounding syntax does.

An object update uses braces instead: { ...item, popular: true } copies fields into a new object and overwrites the popular field. An array of objects often needs both forms, but at different levels. Read the syntax from the outside inward: “new array containing new or existing item objects.” This is more reliable than remembering a string of punctuation.

```
// Standalone TypeScript example.
const oldNames = ['Tea', 'Rice'];
const newNames = [...oldNames, 'Soup'];
console.log(oldNames); // ['Tea', 'Rice']
console.log(newNames); // ['Tea', 'Rice', 'Soup']
const oldItem = { name: 'Tea', popular: false };
const newItem = { ...oldItem, popular: true };
console.log(oldItem.popular); // false
console.log(newItem.popular); // true
```

References explain why mutation is dangerous

An array variable holds a reference to an array object. If you call menu.push(item), you change that same object. Passing the same reference back to the setter can be treated as unchanged because React compares

state with Object.is. The deeper problem is that you have also changed data belonging to an earlier render. Another component or closure may still refer to that object.

A new array gives React a new reference and preserves the old array as a snapshot. This is not a claim that React can never render after mutation: some unrelated update might cause a render and reveal the mutation later. That makes mutation bugs confusing. The reliable rule is to avoid modifying arrays and objects already stored in state.

Common trap: push returns the new numeric length of the array. setMenu(menu.push(item)) both mutates the old array and passes a number where an array is expected.

Append and prepend with an updater

When an operation depends on previous state, use the updater form. React supplies the pending array to your function, and your function returns a fresh array. The name previous is just a parameter name, not a special React keyword. You can call it current or items, but choose a name that makes the data flow obvious.

Keep the updater pure. Construct a new item, including any generated identity, in the event handler before passing it into the updater. The updater should only calculate the next array. This keeps the code safe if React invokes an updater again during development checks.

```
// Alternative fragments inside a component, with item: MenuItem.
setMenu(previous => [...previous, item]); // append
setMenu(previous => [item, ...previous]); // prepend
setMenu([]); // clear: the next state is known directly
```

Understand slice by following the boundary

slice(start, end) copies a segment without changing the source. The end position is excluded. To insert an item at index 2, combine the part before index 2, the new item and the part beginning at index 2. To replace index 2, skip that old element by starting the suffix at index 3. To delete it, keep both surrounding pieces and omit the replacement.

Write down the indexes for a small example before coding. For [A, B, C, D], index 2 refers to C. The prefix slice(0, 2) contains A and B. The suffix slice(2) contains C and D, while slice(3) contains only D. The one-character difference determines whether you insert or replace. splice has a similar name but mutates the original array.

```
// Standalone TypeScript examples.
const values = ['A', 'B', 'C', 'D'];
const inserted = [...values.slice(0, 2), 'X', ...values.slice(2)];
const replaced = [...values.slice(0, 2), 'X', ...values.slice(3)];
const removed = [...values.slice(0, 2), ...values.slice(3)];
// inserted: A B X C D; replaced: A B X D; removed: A B D
```

Copy the changed object as well as the array

Spreading an array is a shallow copy. The new array initially contains references to the same item objects. If you create copy = [...menu] and then set copy[0].popular = true, you have still changed the original first object. Copy the object you change, and reuse unchanged objects.

map is useful for replacing one item by ID. filter is useful for removing an item by ID. An ID-based update describes the item you mean even if its position has changed. The ternary below returns a new object for the matching item and the existing object for all others. React gets a new array, and the changed item gets a new object reference while keeping its stable id.

```
// Fragments inside a component; targetId is a string.
setMenu(previous => previous.map(item =>
  item.id === targetId ? { ...item, popular: !item.popular } : item
));
setMenu(previous => previous.filter(item => item.id !== targetId));
```

Pause and predict: Does [...menu] make an independent deep copy of every MenuItem?

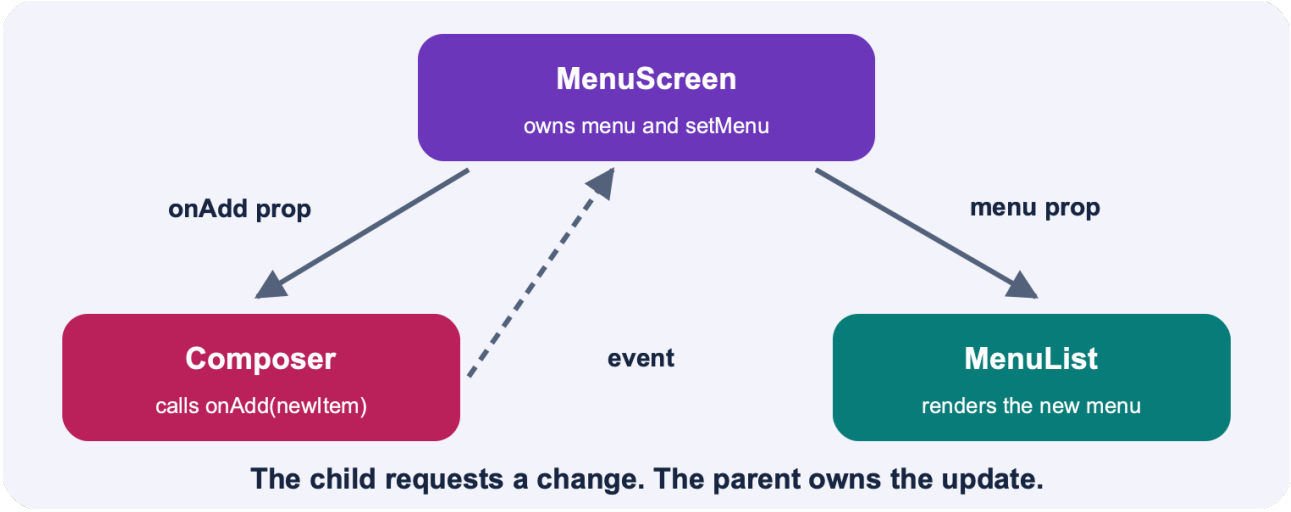
Explanation: No. It copies the array, but each element still refers to the same object. Create a new object for any item whose fields you change.

Read more: React Updating Arrays in State / React useState

Connect a form to its parent with callback props

Essential / Lecture 3b slides 15-16; Lecture 4b slides 10-17

- Choose the component that owns shared state.
- Control TextInput with value and onChangeText.
- Trace an event from child to parent and the updated data back down.



MenuScreen owns menu state. It passes rows to MenuList and a callback to Composer. Composer calls that callback; the parent updates state and renders again.

Put shared state at the nearest common owner

Suppose one component displays a menu and another component creates a new item. Both need access to the saved menu, so keep that array in their closest common parent. The parent passes data down through props. It also passes a function that a child may call to request a change. The child does not directly assign to the parent's state.

This is called lifting state up. It avoids two separate arrays that can disagree about what was added. A child may still own draft text if no sibling needs it. The question is not "Must all state be at the top of the app?" It

is “Which components must agree on this value?” Choose the lowest common owner that can coordinate them.

A controlled input displays the state you give it

TextInput uses value for the displayed string and onChangeText for a callback receiving the new string. In a controlled input, the value comes from React state. When the user types, the callback requests a state update. The later render passes the updated value back into TextInput. If you provide value but never update it, typing may appear to be ignored because the displayed value remains the old string.

onChangeText is especially convenient because it passes text directly. It differs from onChange, which supplies an event object. Do not write event.target.value when using React Native onChangeText. If you need to trim a submitted name, do that at submission time; trimming on every keystroke can make ordinary typing awkward.

```
// Complete small React Native component file.
import { useState } from 'react';
import { Text, TextInput, View } from 'react-native';
export default function DraftName() {
  const [name, setName] = useState('');
  return (
    <View>
      <Text>Item name</Text>
      <TextInput value={name} onChangeText={setName}
        placeholder="For example, Tea" accessibilityLabel="Item name" />
      <Text>You typed: {name}</Text>
    </View>
  );
}
```

Trace the values

- User types a character.
- TextInput calls onChangeText with the new string.
- setName requests a render with that string.
- The next render passes name back through value.

Define the callback contract before its implementation

For a child that collects a name and a popular flag, use an onAdd callback that accepts those fields. This guide calls that input MenuDraft. The parent can add an id later when it creates the saved MenuItem. Separating the draft from a saved item is a useful design choice. An exercise may instead ask the child to provide the whole MenuItem.

The function prop type is the agreement between parent and child. If onAdd expects one object, the child calls onAdd({ name, popular }); it does not call onAdd(name, popular). The braces construct one argument containing two fields. TypeScript can catch this mismatch before you press the button.

```
// Type declarations for the next example.
type MenuDraft = { name: string; popular: boolean };
type ComposerProps = { onAdd: (draft: MenuDraft) => void };
```

Build a child that requests a change

The complete child below owns its draft string. It trims the name before submitting, ignores an empty draft, calls the parent's onAdd function and clears the field. The button receives submit itself, so submission happens after a press. A more complex child could add a Switch for popular, but the callback relationship would be the same.

The parent supplies the actual implementation of onAdd. This is why the child can be reused: one parent might add a menu item, while another might validate or log the draft. In a live interview, explain both sides. Saying "This button adds the item" is less precise than saying "This button calls the prop; the parent callback updates the array."

```
// Complete reusable child component file.
import { useState } from 'react';
import { Button, TextInput, View } from 'react-native';
type MenuDraft = { name: string; popular: boolean };
type ComposerProps = { onAdd: (draft: MenuDraft) => void };
export default function Composer({ onAdd }: ComposerProps) {
  const [name, setName] = useState('');
  const submit = () => {
    const trimmedName = name.trim();
    if (trimmedName.length === 0) return;
    onAdd({ name: trimmedName, popular: false });
    setName('');
  };
  return (
    <View>
      <TextInput value={name} onChangeText={setName}
        accessibilityLabel="New item name" />
      <Button title="Add item" onPress={submit}
        disabled={name.trim().length === 0} />
    </View>
  );
}
```

Follow the full round trip

In the parent fragment below, the ID is supplied as part of an existing complete MenuItem. The key point is where the handler lives: it is defined in the component that owns menu. The parent gives the handler to a child as a prop. When the child calls it, the parent requests a new array, re-renders and supplies the updated list to its display children.

“One-way data flow” does not mean a child can never send information to a parent. A child can call a callback with a value. The parent decides what state update follows, and the new state then flows down as props. A local state update in a child does not automatically re-render its ancestors. Avoid turning the lecture’s tree diagram into an absolute rule about all rendering; context and other subscriptions can also cause updates.

```
// Parent fragment; MenuItem is defined and useState is imported.
const [menu, setMenu] = useState<MenuItem[]>([]);
const addMenuItem = (item: MenuItem) => {
  setMenu(previous => [...previous, item]);
};
// Child receiving a complete MenuItem callback:
// <ItemPicker onAdd={addMenuItem} />
// Child event handler:
// onPress={() => onAdd({ id: 'tea', name: 'Tea', popular: false })}
// In a real list, enforce unique IDs if an item can be added repeatedly.
```

Pause and predict: Where should saved menu state live if a Composer adds items and a MenuList displays them?

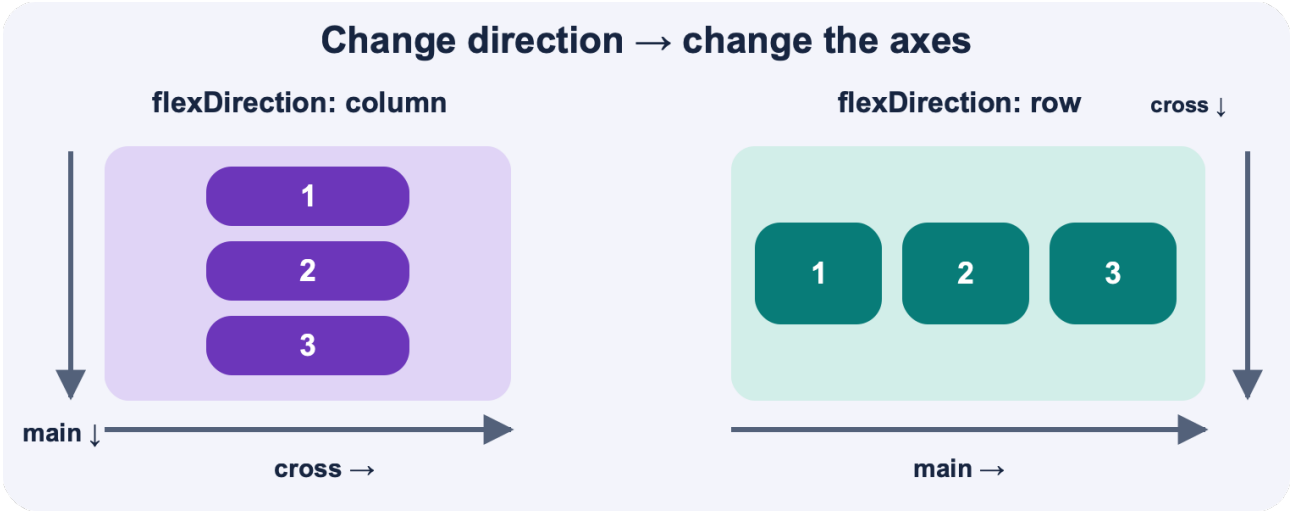
Explanation: In their closest common parent. The parent passes the array to MenuList and an add callback to Composer. Draft input state may remain inside Composer if only Composer needs it.

Read more: React Sharing State Between Components / React Native TextInput / React Responding to Events

Reason about spacing and Flexbox axes

Reinforcement / Lecture 2a slides 22-39; Lecture 2b slides 4-10; Lecture 4a slide 8

- Distinguish padding, border and margin.
- Identify the container and direct children for each layout.
- Predict main-axis and cross-axis alignment in a row or column.



In a column, main axis points down and cross axis points across. In a row, main points across and cross points down. Justify content follows main; align items follows cross.

Style objects describe each component

React Native styles are JavaScript objects, usually written with camelCase property names such as backgroundColor and marginTop. StyleSheet.create groups named style objects and helps the type checker

recognize supported properties. You apply one style with style={styles.card}. An inline object such as style={{ padding: 12 }} is also valid.

You can combine styles in an array. Later entries override earlier values for the same property. This is useful for a selected or disabled state without copying every base style. Text styles, such as fontSize and color, belong on Text. A container's text style does not work like a browser CSS rule that automatically cascades to all nested Text components.

```
// Fragment with StyleSheet imported from react-native.
const styles = StyleSheet.create({
  card: { padding: 16, backgroundColor: '#EAF3FF', borderRadius: 12 },
  selected: { backgroundColor: '#C7F5DC' },
  title: { fontSize: 20, fontWeight: '700', color: '#10233F' },
});
// JSX fragment: <View style={[styles.card, selected && styles.selected]} />
```

The box model explains where space goes

Content sits inside padding. The border surrounds the padded content. Margin creates space outside that border. If a card's background should include the space around its text, use padding. If two cards need a gap between their outer edges, use margin or a container gap. These choices can look similar with no background, so draw the colored boxes when you are uncertain.

Border styling uses properties such as borderWidth, borderColor and borderRadius. For a particular edge, use complete React Native property names such as borderTopWidth or borderLeftColor. A shorthand name from a slide or web CSS example may not be a valid React Native style. Numeric dimensions are density-independent units, not CSS strings ending in px.

Pause and predict: A card background should extend 16 units past its text on every side. Is that margin or padding?

Explanation: Padding. It lies inside the card border and background. Margin would add space outside the card.

Identify the main axis before naming an alignment

Flexbox controls the direct children of a container. flexDirection chooses the main axis. In React Native the default is column, so the main axis is vertical and the cross axis is horizontal. With flexDirection: "row", the main axis is horizontal and the cross axis is vertical. justifyContent distributes children along the main axis. alignItems aligns them along the cross axis.

Do not memorize "justifyContent means horizontal" because it fails as soon as direction changes. Instead say, "This is a row, so justifyContent controls horizontal distribution." center packs children toward the center; space-between leaves space between the first and last items while placing them at the ends of the available main-axis space. alignSelf lets an individual child override the container's cross-axis alignment for itself.

```
// Style object for a horizontal row of items.
const rowStyle = {
  flexDirection: 'row' as const,
  justifyContent: 'space-between' as const,
  alignItems: 'center' as const,
  padding: 16,
};
```

Size the container before distributing free space

Alignment becomes visible only when the container has space to distribute. flex: 1 commonly lets a child expand into available space within a sized parent. Fixed numeric dimensions give a specific size; percentage dimensions depend on the parent's size. A child with height: "100%" cannot invent a useful height if the parent has no resolved height.

In a row with equally eligible children, flex: 1 and flex: 2 can allocate remaining space in a one-to-two ratio, but explicit dimensions, minimum sizes, content and other flex rules can affect the result. React Native also differs from web defaults: flexDirection starts as column and flexShrink defaults to 0. Long content can therefore overflow unless you provide appropriate wrapping, shrinking or scrolling.

The width examples in Lecture 2b mean different things: the default comes from normal layout rules, width: 200 requests a fixed width, width: "100%" uses the resolved parent width, and width: windowWidth uses a measured window width. A parent may be narrower than the window because of padding or a sidebar, so percentage and window-based widths are not interchangeable.

```
// JSX fragment; View and Text are imported.
<View style={{ flex: 1, flexDirection: 'row' }}>
  <View style={{ flex: 1, backgroundColor: '#C7F5DC' }}>
    <Text>One share</Text>
  </View>
  <View style={{ flex: 2, backgroundColor: '#D9DEFF' }}>
    <Text>Two shares</Text>
  </View>
</View>
```

Build a two dimensional screen from nested groups

For a profile screen, start with a vertical page container. Put the avatar and name in a horizontal row. Inside the name area, use another vertical container for title and subtitle. Put action buttons in a separate row. Each container controls its own direct children, so you can solve one small alignment problem at a time.

Before coding, sketch boxes and label each group row or column. Identify which group expands and where scrolling belongs. ScrollView needs a bounded height, commonly obtained by making its ancestors fill the screen. Use contentContainerStyle to style the scrollable content inside it and style to style the ScrollView itself. This distinction is useful when content padding or minimum height seems to have no effect.

Check dimensions on a changing screen

Lecture 2b introduces Dimensions.get("window").width. That reads the current window dimensions at that moment. A layout calculated once at module scope may not react to rotation, split-screen resizing or font changes. For a component that must respond to window changes, useWindowDimensions is a useful official React Native hook.

Prefer flexible layout for ordinary rows and columns instead of manually calculating every coordinate. Verify the screen at a narrow width and with a long label. Cross-platform layout is not finished when it looks correct in one browser preview. Text metrics, safe areas, keyboards and available space can differ across devices.

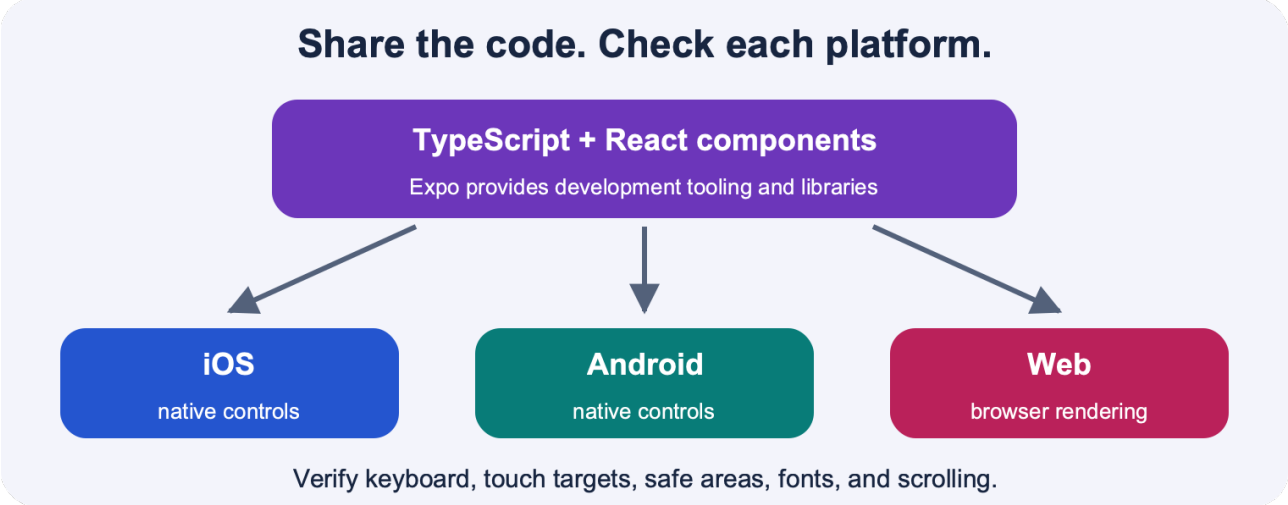
```
// Fragment inside a component; import useWindowDimensions from react-native.
const { width } = useWindowDimensions();
const compact = width < 380;
// Example JSX: <View style={{ padding: compact ? 12 : 24 }} />
```

Read more: React Native Style / React Native Layout with Flexbox / React Native Height and Width / React Native Images

Know what React Native and Expo provide

Reinforcement / Lecture 1a slides 12-27, 30; Lecture 1b slides 5-8

- Compare web, PWA, native and cross-platform approaches.
- Explain the distinct jobs of React, React Native and Expo.
- Plan a small cross-platform check instead of assuming identical behavior.



Shared TypeScript logic and React components can target iOS, Android, and web. Platform-specific behavior and testing remain necessary.

Separate the three layers

React supplies the component model, props, state and hooks. React Native connects that model to platform UI and device capabilities. Expo adds a framework, libraries and development tools around React Native. When you write useState, that is a React feature. When you render a native TextInput, that is a React Native component. When you use an Expo development workflow, Expo helps you run and build the application.

The course uses Expo to reduce setup work so that you can focus on application behavior. Expo Snack is a convenient browser-based place to experiment, while an Expo project on your computer gives you files and development tooling. Expo Go can run projects that fit the native capabilities bundled into that app. A development build is needed when your project requires native capabilities beyond that bundled set.

Compare approaches using requirements

A browser-only web app is reachable through a URL and can be updated on the server. A progressive web app adds installability and selected platform integration where supported. A fully native app uses each platform's own development stack and can access its APIs directly. Cross-platform frameworks let teams share substantial code while producing applications for more than one platform.

These are tradeoffs rather than a simple quality ranking. A campus information page may work well as a web app. An application needing specific hardware integration or demanding platform behavior may require

native capabilities. Shared code reduces duplicated implementation, but teams still need platform testing and may need platform-specific code. Capabilities of PWAs and frameworks change over time, so avoid memorizing a fixed list of everything a browser can never do.

Pause and predict: Does cross-platform mean every feature works identically on iOS, Android and web?

Explanation: No. It means substantial code can be shared. Platform APIs, interaction conventions and component support can differ, so you still test the platforms you intend to support.

React Native is not just a webpage inside a phone

The lecture groups several technologies under a broad cross-platform or hybrid category. For precision, React Native's usual iOS and Android UI is built from native platform views; it is not simply an HTML page placed inside a WebView. Other cross-platform approaches have different rendering strategies. React Native web support maps components to web behavior through additional tooling.

This explains why browser HTML and CSS examples need translation. A div becomes a View, visible text belongs in Text, and the style system supports React Native properties rather than every browser CSS feature. It also explains why browser preview is useful but not sufficient evidence that a mobile interaction works correctly.

Test the parts users can feel

For a small practice screen, test text entry, keyboard behavior, scrolling, button presses, long labels and narrow layouts. Add clear accessible labels to custom controls and preserve readable contrast. A pressable region should have enough space to activate comfortably. Check that disabled controls communicate why they are unavailable through surrounding text or labels when the reason is not obvious.

Use a consistent primary platform during timed practice so that setup does not consume your interview time. Then run the same app on another supported platform and explain any differences. This gives you practical evidence for a cross-platform discussion while reinforcing the central syntax and state skills.

Make a minimal app run before expanding it

Open the course's existing Expo environment or an appropriate TypeScript Snack. Verify a View and Text render. Add a button that changes a number. Only then add the list and child input. This creates a known working baseline and makes each new error easier to locate.

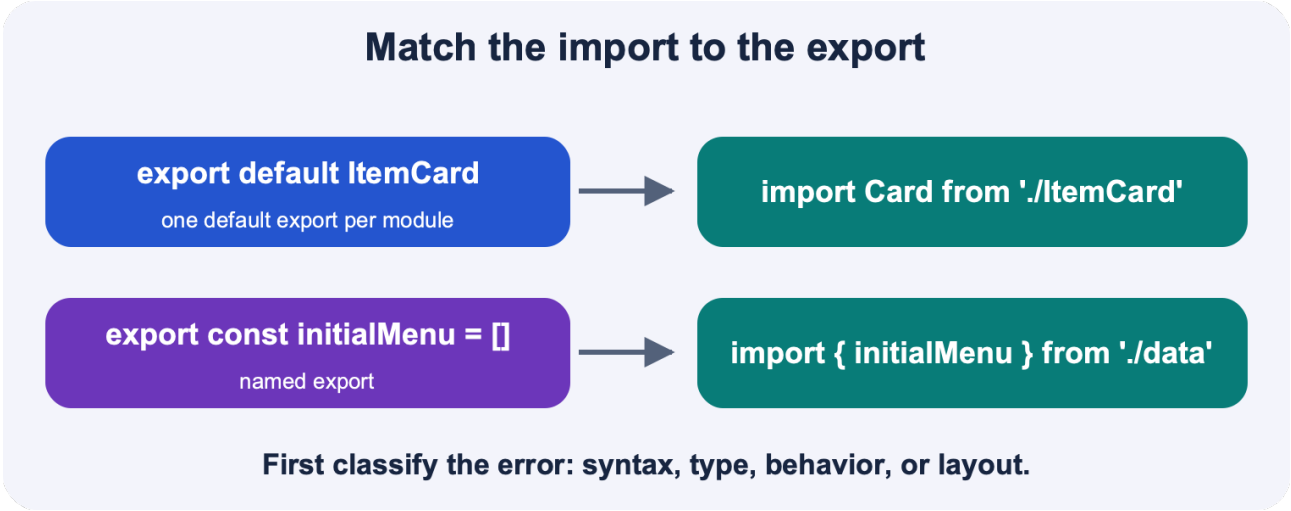
Package versions and Expo SDK compatibility change. Follow the setup instructions for the actual course project rather than upgrading its dependencies immediately before an interview. Expo's npx expo install selects package versions intended to work with the project's Expo version. Knowing what the tool does is more valuable here than memorizing a global installation command.

Read more: React Native Core Components and Native Components / React Native Platform Specific Code / Expo Tutorial Using React Native and Expo

Organize files and recover from errors aloud

Reinforcement / Lecture 3a slides 13-20; Lecture 3b slides 17-24; Lecture 4a slides 5-11, 17-24

- Match named and default imports to exports.
- Read the first useful error before changing code.
- Demonstrate an independent problem-solving process in 25 minutes.



A default export is imported without braces and may be renamed. A named export is imported with braces and its exported name, unless aliased.

Match imports to the export form

A file can have one default export and many named exports. Import a default export without braces; the importing file chooses its local name. Import a named export with braces and its exported name, unless you explicitly rename it with as. A package import names an installed dependency, while a relative import such as ./ItemCard names a local file.

When a component cannot be found, inspect both files. Is the component exported? Is the import named or default? Does the path point to the correct location and match capitalization? These checks are often more useful than editing JSX. An unresolved module error is a different problem from passing the wrong prop type.

A named import may be renamed locally with as: import { colors as palette } from "./colors". A namespace import gathers a module's exported names under one object: import * as theme from "./colors" lets you access theme.colors. The import screenshots in the lecture show these alternatives. Changing the local name does not change the original export, and namespace imports do not make every imported value a default export.

```
// File: types.ts
export type MenuItem = { id: string; name: string; popular: boolean };

// File: ItemCard.tsx (outline of the export form)
// export default function ItemCard(...) { ... }

// File: App.tsx
import ItemCard from './ItemCard';
import type { MenuItem } from './types';
import { useState } from 'react';
import { Text, View } from 'react-native';

// Alternative forms for a module exporting a named colors value:
// import { colors as palette } from './colors';
// import * as theme from './colors'; // then use theme.colors
```

Classify the error before fixing it

A syntax error means the language could not parse the code: perhaps a tag is unclosed or a block is missing its closing brace. A type error means the code's described values do not fit: perhaps an object is supplied where an array is expected. A runtime error occurs while the program runs, such as reading a field of undefined. A behavior bug can have no error message at all, such as a button that runs too early or a list that does not update.

Read the first useful error, locate the expression it points to and compare the expected and actual shape. Later errors can be consequences of the first one. Change one thing, run again and describe what changed. If you randomly replace several brackets, you lose evidence about which assumption was wrong.

Pause and predict: TypeScript says a number is not assignable to MenuItem[]. What should you inspect first?

Explanation: Inspect the expression being passed as the next array state. A common cause is menu.push(item), which returns a number rather than a new array.

Use a short verbal debugging loop

Say what you expected, what you observed and one hypothesis that could explain the difference. For example: "I expected a new row after pressing Add. The handler logs the draft, so the press works. I will inspect whether the setter returns a new array." This shows the interviewer a reasoning process even while the code is incomplete.

Test the smallest part that can settle the hypothesis. If the callback is never called, do not spend time on array spread yet. If the callback is called but the array stays unchanged, inspect the update. If the array changes but no row appears, inspect map and rendering. Every step should narrow the possibilities.

Prepare to work within the course assessment rules

Lecture 4a describes the initial coding interviews as 25-minute live programming tasks in Expo Snack, with a reference screenshot and a staged handout. It says the process matters and students are not expected to complete every part. These slides concern the first coding interview; they are useful practice context, but they do not establish the exact content or rules of every later assessment.

The same slides permit general internet reference and, optionally, one page of handwritten English notes on both sides. They prohibit other AI tools apart from the stated search-summary exception and prohibit homework solutions or solution-like code, including handwritten copies. Treat this study kit as preparation, not an authorized interview aid. Confirm current assessment instructions with the course staff; do not carry its code-rich cheat sheet into the assessment as if it were approved.

Allocate practice time to the observed gaps

Spend the first few minutes reading requirements and identifying the data shape, state owner and repeated component. Build the smallest visible screen, then render initial data. Add a callback that performs one immutable update. Add the controlled input and connect it to the callback. Use the final minutes to test an empty input, a second addition and any requested condition.

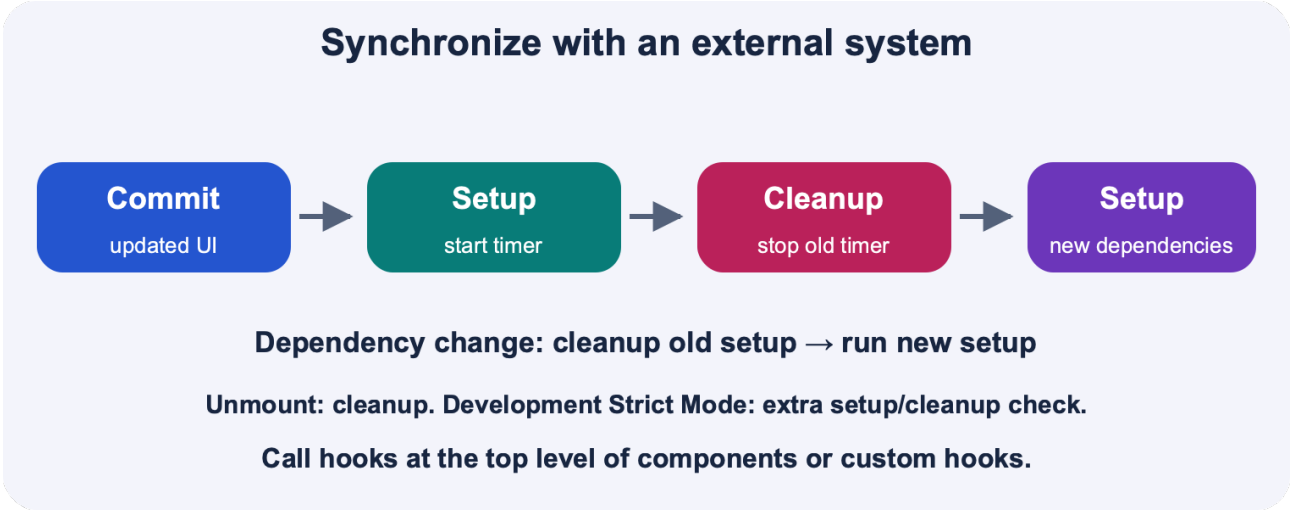
This sequence gives the core interaction enough practice time. Do not spend the first half of practice making the screen visually perfect while the parent-child interaction remains unimplemented. After each mock, write one sentence about the first place you became uncertain. Then isolate that operation for a five-minute drill before doing another full mock. A change of nouns in the next task helps reveal whether you learned the concept or only the previous answer.

Read more: TypeScript Modules / Expo Using TypeScript

Use hooks and timers without creating hidden repeats

Later lecture breadth / Lecture 5a slides 3-14; Lecture 5b slides 3-6

- Keep hook calls at the top level in a stable order.
- Choose an effect only for synchronization with an external system.
- Explain dependencies, cleanup and stale state in a timer.



After a commit React runs setup. When dependencies change, cleanup runs before the new setup. Unmount also cleans up. Strict Mode can add a development-only setup cleanup setup check.

Hooks connect a component to React features

useState and useEffect are hooks. For these hooks, call them at the top level of a function component or a custom hook, in the same order each render. Do not put them in a loop, event handler or conditional branch. If a component sometimes returns early, place its hooks before that conditional return so that their order remains consistent.

The lecture says hooks must be called exactly once every render. The practical rule is more precise: each hook call site must occur in a consistent order for that component's renders. A component may use several useState calls, and React may render the component more than once. A function name starting with use is a convention for hooks, not a magic permission to call it anywhere.

```
// Fragment inside a component. Hooks remain unconditional.
const [running, setRunning] = useState(false);
const [seconds, setSeconds] = useState(0);
useEffect(() => {
  if (!running) return;
  // Start external synchronization here.
}, [running]);
```

An effect synchronizes after a committed render

An effect is appropriate when your component must keep an external system in step with its state, such as starting and stopping an interval. A button's direct response belongs in its event handler. A value derived from other state usually belongs in a calculation during render. You do not need an effect to compute menu.length or to append an item immediately after a press.

Timer callbacks execute later, after the current JavaScript work has finished and the delay allows them to run. setTimeout schedules one callback; setInterval repeatedly schedules callbacks until cleared. Delays use milliseconds, and actual execution can be later than the requested delay. A one-second interval is adequate for a learning counter but is not a guarantee of precise elapsed time.

Read the dependency list as values React compares

With no dependency argument, an effect runs after each commit of that component. With [], it sets up for a mount without re-running because of changed reactive dependencies. With [running], it also re-synchronizes after running changes. React compares each dependency with its previous value using Object.is; a newly created array containing the same dependency values does not by itself mean those values changed.

List the reactive values that the effect reads. Do not choose [] merely to silence repeated runs if the effect actually depends on a changing prop. An effect that sets state on every render can cause a loop. The right fix is to understand what the effect synchronizes and when it should restart, not to suppress a warning blindly.

Pause and predict: If an effect depends on [running], should changing an unrelated draft string restart its interval?

Explanation: No. If running is unchanged, that dependency has not changed. A render caused by draft text alone does not require this effect to re-synchronize.

Pair setup with cleanup

The cleanup function undoes the resource established by that effect setup. For a timer, retain the ID returned by setInterval and return a function that calls clearInterval. React calls cleanup before the next setup when dependencies change and when the component is removed. An interval created in the component body would be created again on every render, producing multiple independent timers.

In development Strict Mode, React can perform an extra setup-cleanup cycle as a check. Therefore “[] means exactly once forever” is not a safe mental model. Cleanup should make repeated setup and teardown harmless. The example below needs only running as a dependency because the functional updater reads pending seconds, not a seconds value captured by the effect.

```
// Complete React Native learning timer component.
import { useEffect, useState } from 'react';
import { Button, Text, View } from 'react-native';
export default function Timer() {
  const [running, setRunning] = useState(false);
  const [seconds, setSeconds] = useState(0);
  useEffect(() => {
    if (!running) return;
    const id = setInterval(() => {
      setSeconds(previous => previous + 1);
    }, 1000);
    return () => clearInterval(id);
  }, [running]);
  return (
    <View>
      <Text>{seconds} ticks</Text>
      <Button title={running ? 'Pause' : 'Start'}
        onPress={() => setRunning(value => !value)} />
    </View>
  );
}
```

Trace the values

- Initially running is false, so no interval is started.

- Press Start: running becomes true and the effect starts one interval.
- Each tick computes the next seconds from pending state.
- Press Pause: cleanup clears the old interval; the new setup starts none.
- Unmount: React also cleans up any active interval.

Recognize a stale closure

A callback keeps access to the values from the render in which it was created. If an interval created with [] repeatedly calls setSeconds(seconds + 1), and seconds was 0 in that render, it repeatedly requests 1. It does not automatically see future seconds values. The updater setSeconds(previous => previous + 1) asks React to supply pending state instead.

This connects the later lecture to the earlier state lessons. Functional updates help express state changes safely when callbacks run later or several updates are queued. Practice explaining this connection after the core menu problem is comfortable. The supplied decks do not establish the exact questions in any particular assessment.

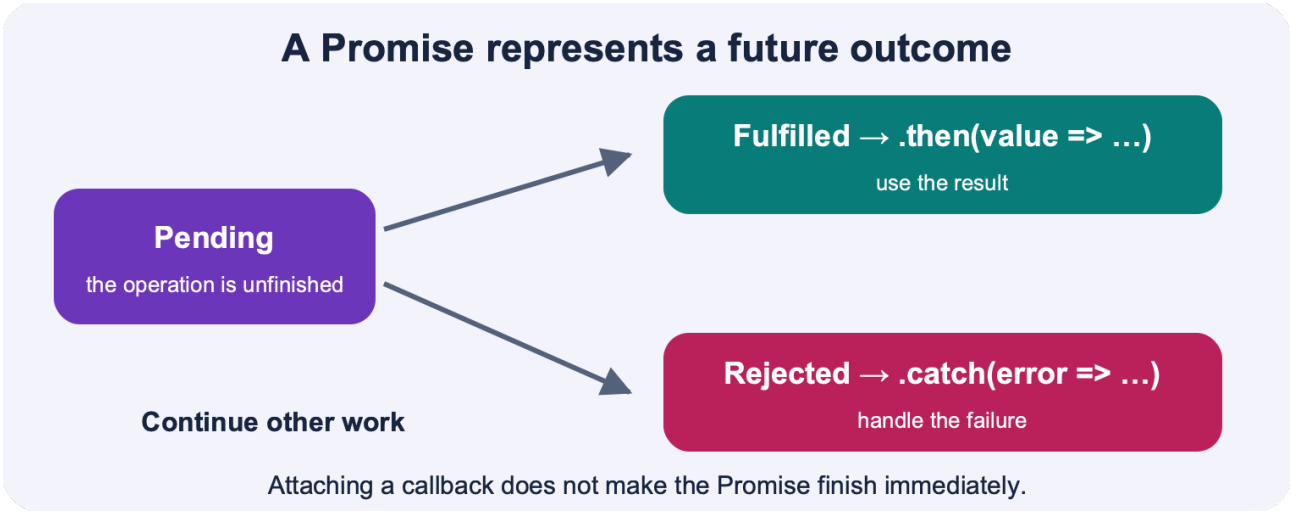
Common trap: An effect setup callback must not be an async function, because an async function returns a Promise rather than a cleanup function. If you need asynchronous work, define and call an inner async function and keep cleanup synchronous.

Read more: React useEffect / React Rules of Hooks / React Native Timers

Follow asynchronous results without blocking the screen

Later lecture breadth / Lecture 5b slides 7-13; Lecture 5a slides 4-6

- Describe pending, fulfilled and rejected Promise states.
- Predict the ordering of immediate code and then callbacks.
- Return chained work and handle failure deliberately.



An operation starts pending and eventually fulfills with a value or rejects with an error. Then handles fulfillment; catch handles rejection.

A Promise represents a result that may arrive later

A Promise is an object representing the eventual outcome of an operation. It starts pending and becomes fulfilled with a value or rejected with a reason. Once settled, its outcome does not change. You attach a fulfillment callback with then. The callback receives the fulfilled value; the Promise itself is not that value.

This extends the callback ideas you already know. A timer accepts a callback when you schedule it. A Promise-returning operation gives you an object to which you attach a callback. Writing .then does not pause the surrounding function. Code after the registration continues, and the callback runs when the Promise can deliver its outcome.

Predict the order of synchronous and queued work

In the example, A prints immediately. Promise.resolve creates an already-fulfilled Promise, but its then callback is still scheduled to run after the current synchronous code. C therefore prints before B. This matters when you try to assign an asynchronous result to a local variable and use it immediately outside the callback: the value may not have arrived yet.

To use the result in a screen, request a state update inside the fulfillment callback. React can then render the later value. A console log immediately after starting the operation is not evidence that the operation has already finished. Name the moment when each line can run.

```
// Standalone JavaScript or TypeScript example.
console.log('A');
Promise.resolve('B').then(value => console.log(value));
console.log('C');
// Output: A, C, B
```

Pause and predict: What does .then(callback) return?

Explanation: It returns a new Promise. The value returned by callback determines that new Promise's fulfillment value, or its eventual outcome if callback returns another Promise.

Return the result that the next step needs

A then callback can transform the result. If it returns a normal value, the next then receives that value. If it returns another Promise, the chain waits for that Promise's outcome. If the callback has a block body and forgets return, the next step receives undefined. This is the same expression-body versus block-body issue that caused trouble in map.

The lecture focuses on attaching callbacks and receiving results. Rejection handling and chaining are useful related extensions. A throw in a then callback rejects the resulting Promise, and catch can handle rejection. A catch callback can itself return a recovery value or throw again. These details explain why swallowed errors or missing returns can make asynchronous code appear to succeed with no useful data.

```
// Standalone Promise transformation example.
Promise.resolve(3)
  .then(value => value * 2)
  .then(value => { return Count: ${value}; })
  .then(message => console.log(message)) // Count: 6
  .catch(error => console.error('Could not finish:', error));
```

Model loading, success and failure in the UI

When a result is not yet available, show a loading indicator or message. On success, display the data. On failure, display a useful error state and a deliberate try-again action if appropriate. A Promise rejected by an operation does not automatically create a readable error message on the screen.

For a deterministic practice task, use a local Promise-based delay rather than a real service. This isolates callback timing from network availability. The delay function below is an invented teaching helper; setTimeoutAsync is not a built-in JavaScript API. Lecture 5b similarly introduces a hypothetical Promise-based timer to explain the pattern.

```
// Standalone TypeScript helper, followed by usage.
function waitForLabel(delayMs: number): Promise<string> {
  return new Promise(resolve => {
    setTimeout(() => resolve('Ready'), delayMs);
  });
}
waitForLabel(500).then(label => console.log(label));
console.log('Waiting'); // prints before Ready
```

Know where the provided lectures stop

Lecture 5b announces real fetching as the next topic, but that later lecture is not in the provided folder. Navigation, persistent storage and a full networking lesson are also absent from these ten decks. The core study sequence therefore covers the supplied Promise concepts without pretending these other topics were taught here.

As related background, async/await is another way to consume Promises: an async function returns a Promise, and await pauses that function's continuation while other work can continue. If you later use fetch, check HTTP status and validate the data shape; a TypeScript annotation does not validate a server response. Keep those extensions behind the core goals until you can explain map, state updates and callback props independently.

Common trap: A response to an earlier request can arrive after a newer request. If you extend practice to changing queries, use cleanup or cancellation/ignore logic so stale results do not replace the latest screen. This is beyond the supplied introductory Promise slides.

Read more: Ecma TC39 ECMAScript Promise Objects

Lecture coverage

Lecture 1a

12-27, 30; interview context 34-35

Topics: Native and cross-platform development; Browser apps and PWAs; Framework tradeoffs; React Native ecosystem; Course learning objectives

Slides about staff, enrollment, grading and deadlines are course context, not omitted programming concepts. Cross-platform claims are explained as tradeoffs, not timeless capability limits.

Lecture 1b

5-8

Topics: Expo and Expo Go; Exploring React Native; Core components and APIs

The deck directs learners to a live code activity. Its linked code may add examples beyond the text slides; course research records separately what could be retrieved.

Lecture 2a

4-6, 12-44

Topics: JavaScript and TypeScript; Objects and props; Arrow functions and types; JSX and TSX; StyleSheet; Box model; View and Text; Function declarations

Covers object returns with parenthesized arrow expressions, clarifying the shorthand puzzle on slide 43. Use complete React Native border property names.

Lecture 2b

4-10

Topics: Flex containers and direct children; flexDirection; justifyContent; alignItems; flex and free space; Dimensions; Fixed and percentage sizes

Includes nested layout reasoning and notes current responsive useWindowDimensions as a related extension.

Lecture 3a

5-6, 8-20

Topics: Arrow functions; State across renders; useState; Deferred state updates; Imports and exports

Corrects the sample setter-name mismatch: a declared setCount must be called setCount.

Lecture 3b

9-11, 13-26

Topics: Function types; map; State review; Custom components; Props destructuring; Imports and exports; Conditional rendering; Shopping-cart variations

Activities are addressed through disabled controls, counts, formatting and layout variations; later exercises supply independent practice.

Lecture 4a

5-11, 13-26

Topics: Interview process and resource rules; map; Custom components; Imports and exports; Conditional UI; Clearing array state

Interview logistics here concern the first coding interview; later assessment rules are not inferred as confirmed from these slides.

Lecture 4b

4-8, 10-17

Topics: Arrays and object spread; Immutable updates; slice insertion replacement removal; References; TextInput; Component trees; One-way data flow; Lifting state; Callback props

Core interaction priority. Clarifies that callbacks can carry child information upward while state ownership remains in the parent.

Lecture 5a

4-14

Topics: Asynchronous callbacks; setInterval and clearInterval; Hooks and hook rules; Direct and functional state updates; useEffect; Dependencies; Cleanup and setTimeout

Slide 10 explicitly confirms functional updater support. Development Strict Mode and stale closures are documented clarifications.

Lecture 5b

3-12; boundary on slide 13

Topics: Effects review; Cleanup; Promises; then callbacks; Fulfilled values; Simulated asynchronous timing

Real fetching is announced for the next lecture, not taught in this supplied deck. Rejections and async/await are labeled related extensions.

Scope and source notes

- This is an independently prepared study guide, not an official Stanford assessment or a guarantee of assessment questions.
- The official Exercise 2 Google Doc is publicly linked but its export returned HTTP 401. The prompt and rubric were not read or reconstructed.
- The public 2026 schedule has later material beyond the supplied decks. Navigation, detailed networking, persistence, animations and deployment are supplementary to the supplied lectures and are not confirmed assessment requirements.

- The 2025 course page and historical 2009 CS147L pages were found, but those years were not used to infer 2026 interview requirements. Old CS147L web-development lab content is a different course context.
- Live documentation may use newer package versions than the course starter. The guide should use stable fundamental APIs and keep the installed course environment.
- Course demo destinations were verified; their interactive contents were not run. Some lecture examples exist as images in slides and should be visually inspected in addition to text extraction.
- The supplied Interview 1 rules in Lecture 4a prohibit homework solution-like code and most AI use during the interview. The generated solution material is for preparation; actual assessment rules and scope should be confirmed with staff.

Official sources

These references support the explanations and correct common simplifications. Live documentation may use newer package versions than the course starter. Keep the course environment unless staff direct otherwise.

Stanford HCI CS 147L Autumn 2026 - Current course context and schedule. The public page uses JavaScript; its linked public bundle was inspected because the search index lagged behind.

https://hci.stanford.edu/courses/cs147l/2026/au/

React useState - Confirms that a setter accepts either a next value or a pure updater function; explains snapshots, batching and development checks.

https://react.dev/reference/react/useState

React Rendering Lists - Explains map, explicit returns in block bodies, and stable keys on the outer element returned for each item.

https://react.dev/learn/rendering-lists

React Updating Arrays in State - Provides the immutable update model for adding, removing and replacing items; warns that copying an array is shallow.

https://react.dev/learn/updating-arrays-in-state

React Queueing a Series of State Updates - Explains why repeated replacements based on one render differ from queued updater functions.

https://react.dev/learn/queueing-a-series-of-state-updates

React Responding to Events - Explains passing an event handler rather than calling it during render, and passing handlers as props. Adapt browser onClick examples to React Native onPress.

https://react.dev/learn/responding-to-events

React Sharing State Between Components - Explains lifting shared state to the nearest common parent and passing values and handlers to children.

https://react.dev/learn/sharing-state-between-components

React useEffect - Defines dependencies, cleanup order, external synchronization and the extra development Strict Mode setup/cleanup cycle.

https://react.dev/reference/react/useEffect

React Rules of Hooks - Supports calling useState and useEffect at the top level of components or custom Hooks, before early returns.

https://react.dev/reference/rules/rules-of-hooks

React Using TypeScript - Shows typed component props and state inference; explains when to provide an explicit state type.

https://react.dev/learn/typescript

TypeScript The Basics - Explains compile-time checking and erased type annotations. Type annotations do not validate remote JSON at runtime.

https://www.typescriptlang.org/docs/handbook/2/basic-types.html

TypeScript Everyday Types - Reference for primitive, array, object, optional-property and union types, including useful inference.

https://www.typescriptlang.org/docs/handbook/2/everyday-types.html

TypeScript Type Inference - Explains inference from initializers and contextual types; useful when reducing unnecessary annotations.

https://www.typescriptlang.org/docs/handbook/type-inference.html

TypeScript More on Functions - Explains function type expressions such as (item: MenuItem) => void and typed callback parameters.

https://www.typescriptlang.org/docs/handbook/2/functions.html

TypeScript Modules - Reference for named exports, default exports, relative imports, aliases and type imports.

https://www.typescriptlang.org/docs/handbook/2/modules.html

React Native Core Components and Native Components - Explains how React Native components use native platform capabilities and views. Avoid treating legacy architecture details as current implementation guidance.

https://reactnative.dev/docs/intro-react-native-components

React Native Core Components and APIs - The core-component reference explicitly linked by Lecture 1b, including View, Text, Image, TextInput, Button and ActivityIndicator.

https://reactnative.dev/docs/components-and-apis

React Native Style - Explains JavaScript style objects, camelCase property names, StyleSheet.create and later styles taking precedence in a style array.

https://reactnative.dev/docs/style

React Native Layout with Flexbox - Explains main/cross axes, direction, alignment and flex ratios; React Native defaults include column direction and flexShrink 0.

https://reactnative.dev/docs/flexbox

React Native Height and Width - Explains numeric dimensions, percentages, available parent space and flex sizing.

https://reactnative.dev/docs/height-and-width

React Native TextInput - Defines value and onChangeText, whose argument is the changed text string. Distinguishes defaultValue and onChange.

https://reactnative.dev/docs/textinput

React Native Pressable - Documents onPress and press behavior for custom buttons.

https://reactnative.dev/docs/pressable

React Native ScrollView - Explains the bounded-height requirement and why ScrollView renders all children while FlatList is useful for long lists.

https://reactnative.dev/docs/scrollview

React Native Images - Explains static image sources, URI sources and explicit dimensions for network images.

https://reactnative.dev/docs/images

React Native Platform Specific Code - Explains Platform.select and platform-specific files when shared behavior must differ across iOS and Android.

https://reactnative.dev/docs/platform-specific-code

React Native Timers - Confirms setTimeout, setInterval and their clearing functions exist in React Native. Used for timer API basics only.

https://reactnative.dev/docs/timers

Ecma TC39 ECMAScript Promise Objects - Language specification for pending, fulfilled and rejected promises and queued then reactions; technical verification reference rather than required student reading.

https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-promise-objects

React Native Networking - Explains Fetch, Promise-based responses, async/await and platform networking differences. Fetching is in the public Week 6 schedule, beyond the supplied decks.

https://reactnative.dev/docs/network

Expo Tutorial Using React Native and Expo - Optional official hands-on tutorial for a universal Android, iOS and web app. Its current navigation, gestures and native UI content goes beyond the supplied decks.

https://docs.expo.dev/tutorial/introduction/

Expo Using TypeScript - Explains TypeScript setup and compiler configuration in Expo. Preserve the course project configuration instead of upgrading it during study.

https://docs.expo.dev/guides/typescript/

Optional six-hour route

This is an optional structure for learners who prefer a timetable: 360 minutes, including 45 minutes of breaks. Adapt the blocks to what you need, and pause for classes or travel. Use the skill explorer in index.html whenever you would rather choose a topic directly.

Clock schedule: Math 51 is 10:30-11:30 a.m. and CS106B is 12:30-1:30 p.m. Study windows, including planned breaks, are 7:15-10:20 a.m., 11:30 a.m.-12:25 p.m., and 1:30-3:30 p.m. Split the TypeScript block across the first class; use 12:10-12:25 for lunch. Both timed mocks fit after the second class. The short Study Plan and index.html give the full clock schedule. Preparation and transitions total 30 estimated minutes; final travel and arrival take 15 minutes each. There is no spare time, so shorten optional breadth if these estimates are insufficient.

00:00-00:15 Diagnose and prepare

15 minutes. Close the answers. Write two arrow forms, repair a map with no return, append an item immutably, and sketch parent/child state ownership. Limit each task to three minutes.

Success check: Record the first step you cannot explain. Have a running blank screen.

Open: E01, E02, E04, E07. The HTML schedule has direct links.

00:15-00:55 Arrow functions and map

40 minutes. Use 10 minutes for the arrow explanation and map lab, 20 minutes for E01 and E02, and 10 minutes to rebuild both map forms from memory. Explain JSX braces, return, item, and keys.

Success check: Render a list using both callback styles. Predict the result of a missing return.

Open: E01, E02, Arrow explanation, Map lab. The HTML schedule has direct links.

00:55-01:05 Take a break

10 minutes. Step away from the screen. Do not fill the break with more reading.

Success check: Return ready to focus.

01:05-01:50 Array state and updater functions

45 minutes. Use 10 minutes for the state queue lab, 20 minutes for E04, 10 minutes to test two queued updates, and five minutes to explain direct values versus updater functions.

Success check: Write setMenu(previous => [...previous, item]) and explain every part. Show the valid direct alternative.

Open: E04, State queue lab, Array explanation. The HTML schedule has direct links.

01:50-02:40 Build parent and child callbacks

50 minutes. Trace the callback lab for 10 minutes. Build E07 in 25 minutes. In the final 15 minutes, change the callback from a string to an object with name and popular. Treat provided types as scaffolding until the typing block.

Success check: Explain press -> child handler -> parent callback -> setter -> new render. Distinguish event callbacks from state updaters.

Open: E07, Callback lab, Object callback example. The HTML schedule has direct links.

02:40-02:50 Take a break

10 minutes. Rest your eyes and move. Leave the code closed.

Success check: Resume with a clear head.

02:50-03:15 Add TypeScript deliberately

25 minutes. Use E08 for 15 minutes: infer state from a typed initial array, then type an empty array, a nullable selection, and callback props. Spend 10 minutes explaining which types are inferred and why.

Success check: Distinguish useState(initialMenu) from useState<MenuItem[]>([]) and type a callback without any.

Open: E08, Inference explanation. The HTML schedule has direct links.

03:15-03:45 Connect the minimum menu flow

30 minutes. Use E10 as a checklist with the solution closed. Build only a parent menu, a child form, and mapped cards. Add an item and clear the input. Toggle/remove features and styling are optional.

Success check: Enter a name and popularity, press Add, and see a new card. Explain the complete data flow.

Open: E10. The HTML schedule has direct links.

03:45-04:00 Lunch and break

15 minutes. Use this scheduled break for food and a screen-free pause before class.

Success check: Begin the next block with a fresh file.

04:00-04:25 Run the core mock interview

25 minutes. Start M01 with the 25-minute timer. Keep answers closed, narrate your decisions, and test the add flow. Record any hints after time ends.

Success check: Collect evidence of what you can build independently.

Open: M01. The HTML schedule has direct links.

04:25-04:50 Repair the weakest core step

25 minutes. Score M01 for five minutes using visible behavior. Pick the earliest broken step, repair it for 10 minutes, then close the answer and rebuild it with new data for 10 minutes.

Success check: Explain the cause of one error and demonstrate the corrected skill from memory.

Open: E02, E04, E07, E08. The HTML schedule has direct links.

04:50-05:00 Take a break

10 minutes. Rest before the final transfer attempt.

Success check: Return with enough attention for transfer and recall.

05:00-05:25 Check lecture breadth selectively

25 minutes. If the core add flow works, spend five minutes each on Flexbox axes, imports, cleanup, Promise order, and cross-platform components. If it still fails, use this block for core repair and mark breadth for later.

Success check: Give brief explanations or demonstrate the repaired core flow. This is a survey, not mastery of every later topic.

Open: Q16, Q19, Q22, Q23, Q24. The HTML schedule has direct links.

05:25-05:50 Test transfer with a new prompt

25 minutes. Attempt M02 from a blank file. If the add flow is still unreliable, rebuild that flow using different data instead. Keep answers closed and explain what transferred.

Success check: Build with changed requirements instead of copying a remembered solution.

Open: M02. The HTML schedule has direct links.

05:50-06:00 Finish with closed-reference recall

10 minutes. Write the five core patterns for five minutes. Check the cheat sheet for two minutes, then close it and explain one remaining mistake for three minutes. Stop at the time limit.

Success check: Name the next specific improvement. Do not start a new advanced topic.

Open: Cheat sheet. The HTML schedule has direct links.

If core skills are still weak, use the breadth block for core repair. Full E10 enhancements, M03, and remaining questions are optional after the six-hour route.